Question & Background:

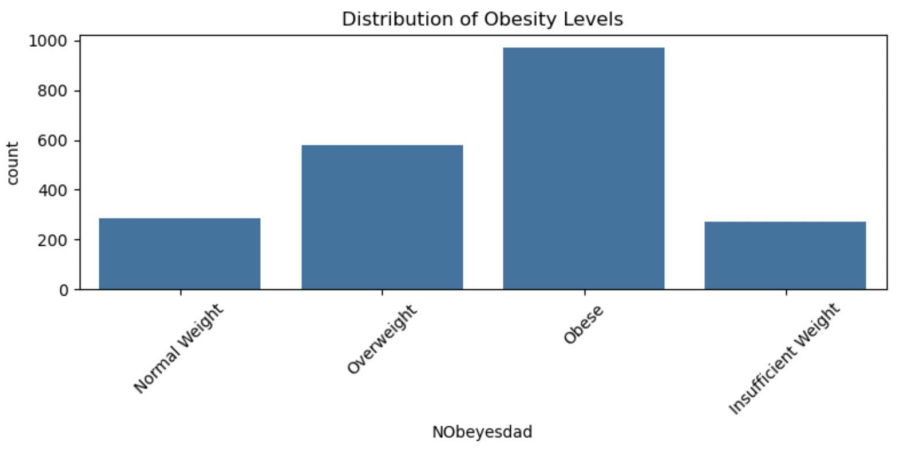
Q: How well can biological characteristics and lifestyle factors predict an individual's weight category?

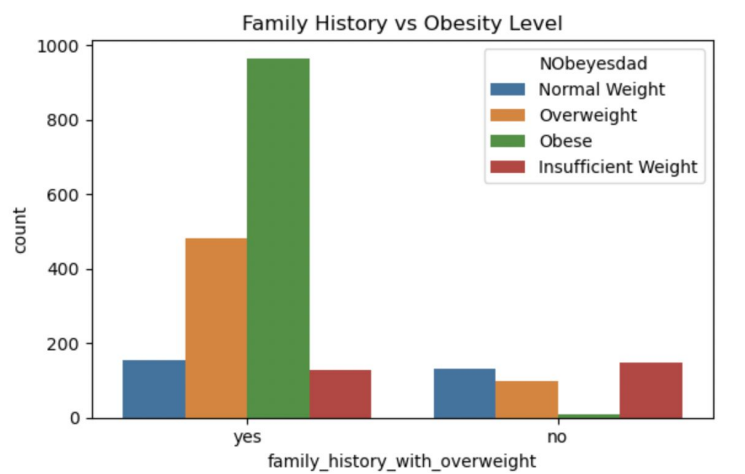
- Method: Classification Decision Tree
- Target variable: **NObeyesdad** (4 levels: Insufficient weight, normal weight, overweight, obese)
- Independent variables: Gender, Age, Height, Weight, family_history_with_overweight, FAVC, FCVC, NCP, CAEC, SMOKE, CH2O, SCC, FAF, TUE, CALC, MTRANS

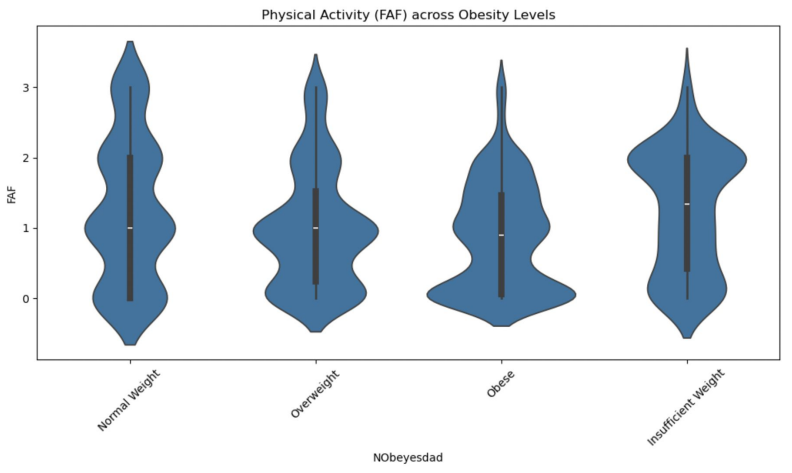
Background:

The CDC identifies physical activity, eating patterns, sleep quality and quantity, and electronic use as key risk factors for obesity. Our dataset, from the class data folder, includes many of these lifestyle-related risk factors as features.

EDA


Distribution of Obesity Levels
count
1000
800
600
400
200
0
Normal Weight
Overweight
Obese
Insufficient Weight
NObeyesdad


Family History vs Obesity Level
NObeyesdad
Normal Weight
Overweight
Obese
Insufficient Weight
count
1000
800
600
400
200
0
yes
no
family_history_with_overweight


Physical Activity (FAF) across Obesity Levels
FAF
3
2
1
0
Normal Weight
Overweight
Obese
Insufficient Weight
NObeyesdad

Method: Preparing + Building the Model

Preparing the Data

- Collapsed the original 7 obesity categories into 4 broader groups for classification: Insufficient Weight, Normal Weight, Overweight, and Obese
- Dropped direct physical variables, like Height and Weight, which overpredict
- Label Encoding to transform all categorical features into numeric format

Building the Model

- Split the dataset into train, tune, test
- Decision Tree Classifier to model non-linear relationships between features and obesity outcomes
- Tuned several hyperparameters and selected the hyperparameter max_depth
- Selected the final model based on the highest F1-macro score to evaluate performance across all obesity categories

Evaluation of the model

Scoring Metrics

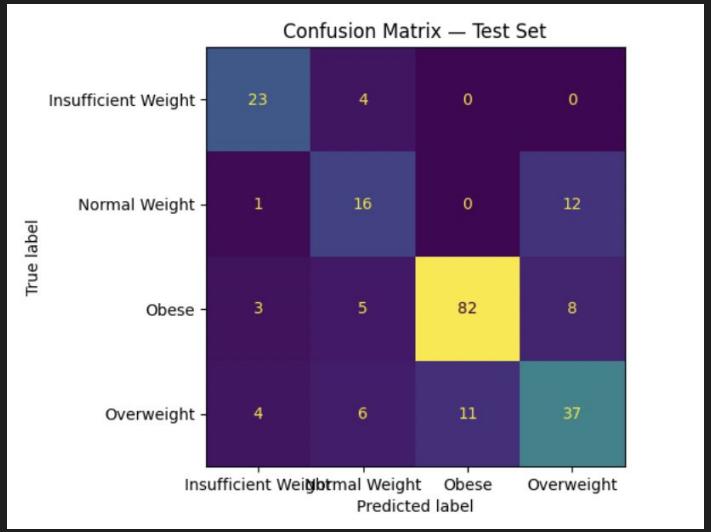
- Used precision, recall, F1-macro score, and balanced accuracy to assess all classes.
  a. precision_macro: checks FP for each class
  b. Recall: identifies how many true positives were identified correctly
  c. f1_macro: balance FP and FN
  d. balanced_accuracy: adjusts for any imbalance between classes
- Focused on F1-macro as the main metric because it balances precision and recall across multiple obesity classes.

Model Results

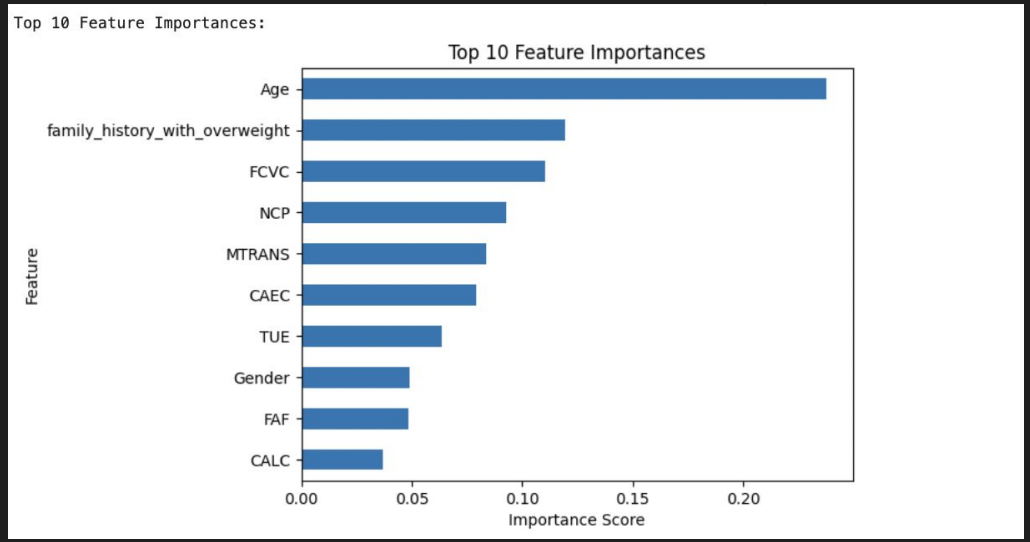
- Slight skew towards false positives for obesity/overweight

Test Set Classification Report:

	precision	recall	f1-score	support
Insufficient Weight	0.74	0.85	0.79	27
Normal Weight	0.52	0.55	0.53	29
Obese	0.88	0.84	0.86	98
Overweight	0.65	0.64	0.64	58
accuracy			0.75	212
macro avg	0.70	0.72	0.71	212
weighted avg	0.75	0.75	0.75	212



Feature Importances



Age

Family history with overweight

FCVC: Frequency of vegetable consumption

NCP: Number of meals per day

MTRANS: Main mode of transportation

CAEC: Consumption of food between meals

TUE: Time spent using electronic devices

Gender

FAF: Frequency of physical activity

CALC: Alcohol consumption

Limitations and Future Work

Limitations of our model

- Self reported data
- Multicollinearity
- Unclear data origin

Improving the model

- Implement ensemble methods with Random Forest, reducing overfitting
- Feature engineering
- Incorporate more meaningful data

Applications

Feature importances can inform public health policymakers on where to focus their efforts.

Method: Why Decision Tree?

- Easy to interpret - helps real-world application for public health information
- Handles numeric and categorical data without much data preprocessing, no need to standardize data
- Cross-validation and hyperparameter tuning helps check overfitting the data
    - Cross-validation: train and test are disjoint, k-fold uses both sets to both train and test

Hyperparameter Tuning

Experimented with:

- **Minimum samples for a node split**
- **Minimum samples for a terminal node (leaf)**
- **Maximum depth of a tree (vertical depth)**
- **Maximum number of terminal nodes**
- **Maximum features to consider for split**

Results

- Best model tuned on max_depth (Macro F1= 0.73)
- We chose Macro f1 as our refit to treat all four obesity classes equally
- Adding min_samples_leaf, min_samples_split, max_features or max_leaf_nodes didn't boost accuracy or F1
- GridSearchCV still picked the max_depth tree as optimal

Takeaway: more hyperparameters didn't improve performance

hyperparameter	accuracy	precision	recall	f1
max_depth	0.75	0.75	0.72	0.73
min_samples_leaf	0.75	0.70	0.72	0.71
min_samples_split	0.75	0.74	0.72	0.73
max_features	0.68	0.64	0.65	0.64
max_leaf_nodes	0.68	0.64	0.65	0.64

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Gender                          2111 non-null   object
 1   Age                             2111 non-null   float64
 2   Height                          2111 non-null   float64
 3   Weight                          2111 non-null   float64
 4   family_history_with_overweight  2111 non-null   object
 5   FAVC                            2111 non-null   object
 6   FCVC                            2111 non-null   float64
 7   NCP                             2111 non-null   float64
 8   CAEC                            2111 non-null   object
 9   SMOKE                           2111 non-null   object
 10  CH2O                            2111 non-null   float64
 11  SCC                             2111 non-null   object
 12  FAF                             2111 non-null   float64
 13  TUE                             2111 non-null   float64
 14  CALC                            2111 non-null   object
 15  MTRANS                          2111 non-null   object
 16  NObeyesdad                      2111 non-null   object
dtypes: float64(8), object(9)
```

	Age	Height	Weight	FCVC	NCP	CH2O	FAF	TUE
count	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000	2111.000000
mean	24.312600	1.701677	86.586058	2.419043	2.685628	2.008011	1.010298	0.657866
std	6.345968	0.093305	26.191172	0.533927	0.778039	0.612953	0.850592	0.608927
min	14.000000	1.450000	39.000000	1.000000	1.000000	1.000000	0.000000	0.000000
25%	19.947192	1.630000	65.473343	2.000000	2.658738	1.584812	0.124505	0.000000
50%	22.777890	1.700499	83.000000	2.385502	3.000000	2.000000	1.000000	0.625350
75%	26.000000	1.768464	107.430682	3.000000	3.000000	2.477420	1.666678	1.000000
max	61.000000	1.980000	173.000000	3.000000	4.000000	3.000000	3.000000	2.000000

Math behind Method

- **Greedy algorithm:** algorithm chooses best split by SSE at that node, doesn't look to future - minimizes gini impurity at every split
- Cross validation - using kfold to cross validate train and test

- Math behind it (Gini Coefficient, Entropy)

gini impurity: how mixed the classes are inside a node

- **split attempts to minimize gini impurity (chance that randomly chosen item is misclassified)**

entropy: amount of uncertainty in a node (pure node=one class: entropy = 0)